

第 3 节（SDD 强化训练）：Sub-Agents 角色分工-三 Agent 协作规格

我们第一周的项目 `ai-knowledge-base/v1-skeleton` 有[三个 Agent](#)：

collector → analyzer → organizer
(采集) (分析) (整理)

当 collector 抛异常时 analyzer 怎么办？analyzer 的输出长什么样 organizer 才能读？这些是 Agent 间的依赖关系和验收标准，用自然语言提示词表达会漏，必须拆成带依赖的显式任务。

本节我们来装 Matt Pocock 的第二个 skill——to-tickets——把高阶 PRD 展开成一组带依赖和验收的任务票（ticket

0，每个 Agent 一份。

时效声明：本节基于 `mattpocock/skills` 2026 年 7 月版。这个 skill 已从 `to-issues` 更名为 `to-tickets`，安装方式、产出路径与字段名都有调整，本节已按新版对齐。开源工具迭代较快，请以仓库 **README** 为准。

方法论（先拆 PRD、再把依赖与验收显式化）是稳定的,工具细节会变。

预备知识

- [老项目逆向 · 改哪里 spec 补到哪里](#)

环境准备

Claude Code

Bash

```
npx skills@latest add mattpocock/skills    # 交互式选择 skill 与 coding agent
```

```
/setup-matt-pocock-skills          # 每个 repo 跑一次,tracker 选  
Local markdown  
ls ~/.claude/skills/    # grill-me / to-tickets 都在
```

OpenCode (推荐)

Plain Text

```
npx skills@latest add mattpocock/skills    # 交互式选择 skill 与 coding agent  
/setup-matt-pocock-skills          # 每个 repo 跑一次,tracker 选  
Local markdown  
ls ~/.agents/skills/
```

本节目标

为 v1-skeleton 的三个 Agent 产出:

1. specs/agents-prd.md —— 一份高阶 **PRD**, 写清 3 个 Agent 的职责和协作流
2. specs/issues/01-collector.md / 02-analyzer.md / 03-organizer.md —— 三份 **issue 任务票**, 每份含 depends_on / acceptance / schema
3. .opencode/agents/{collector,analyzer,organizer}.md —— 三个 Agent 配置文件, 从 issue 派生

双路并行

这个双路并行的设计是为了比较自己和 AI 聊和有 SDD 思想/工具做指导的差异。

A 路 · Vibe · 10 分钟

复制下面的 Prompt 让 AI 做:

Plain Text

我在做 AI 知识库, 有 collector / analyzer / organizer 三个 agent。
帮我写 .opencode/agents/ 下的 3 个角色定义文件。
数据流: collector 抓 GitHub Trending → analyzer 打标签 → organizer 输出

MD。

有什么踩坑点呢？

三个 Agent 各自知道自己要做什么，但谁先谁后、谁触发谁、失败怎么办都在空气里。测试时，你会发现 analyzer 在 collector 还没写完时就启动了。

B 路 · SDD · 40 分钟

阶段 1 · Specify (10 分钟)

先写一份高阶 PRD (specs/agents-prd.md)，强制自己想清“协作”两个字：

Markdown

AI 知识库 · 三 Agent PRD v0.1

总流程

每天 UTC 0:00 触发 · collector → analyzer → organizer · 串行。

Agent 职责

- collector: 抓 GitHub Trending Top 50 · 过滤 AI 相关 · 存 knowledge/raw/
- analyzer: 读 raw · 给每条打 3 维度标签
- organizer: 读已标注 · 整理成 MD

开放问题 (? 用 to-tickets 细化成任务)

- 上游失败下游怎么办？
- 数据怎么传？文件 or 消息？
- 重跑策略？
- 进度追踪？

这里同样故意留下几个？。下阶段让 to-tickets 展开成任务票。

阶段 2 · Clarify (20 分钟)

to-tickets 的作用不是追问（那是 grill-me），是把高阶 PRD 展开成一组带依赖和验收标准的 任务票(ticket)——每个 Agent 一份。

提示词：

Plain Text

使用 to-tickets skill,基于 specs/agents-prd.md 拆成任务票。

产出示例（specs/issues/02-analyzer.md）：

注：新版 to-tickets 的票模板字段为 —— 标题 / What to build(端到端行为)/ Blocked by(阻塞它的票号)/ Acceptance criteria(勾选框)/ Status: ready-for-agent。下方示例保留本书 SDD 结构以便教学，实际跑工具时以上述字段为准。

OC | agents-prd.md 拆分为 is

+

▼

I'll present my proposed breakdown and quiz them per the skill instructions.

我已阅读 PRD。根据 to-issues skill 流程，先提出我的拆分方案供你审查：

Proposed Vertical Slices

#	Title	Type	Blocked by	User stories covered
1	Scaffold & JSON Schema Contracts	HIT L	None	项目骨架、schemas/raw.json + schemas/tagged.json 作为 Agent 间可执行契约、共享 idempotency/—force runner 基座
2	Collector Agent	AFK	#1	抓 GitHub Trending Top 50 → 过滤 AI 相关 → 存 knowledge/raw/{date}.json → 幂等跳过 / --force 重跑
3	Analyzer Agent	AFK	#1, #2	读 raw → 打 3 维标签(tech/innovation/difficulty) → 存 knowledge/tagged/{date}.json → 幂等 + force
4	Organizer Agent	AFK	#1, #3	读 tagged → 整理成 knowledge/articles/{date}.md → 幂等 + force
5	Pipeline Orchestration & Error Handling	AFK	#2, #3, #4	串行编排 collector→analyzer→organizer、上游失败下游跳过、进度追踪、重跑策略

Build DeepSeek V4 Pro DeepSeek

***** esc interrupt

15.9K (2%) · \$0.04 ctrl+p commands

OC | agents-prd.md 拆分为 is x + v

Let me now create the final 3-issue breakdown:

1. Collector Agent + Project Scaffold (AFK) - Sets up project structure, schemas, implements collector

- Blocked by: None

2. Analyzer Agent (AFK) - Implements analyzer

- Blocked by: #1

3. Organizer Agent + Pipeline Wiring (AFK) - Implements organizer, wires pipeline, handles failures

- Blocked by: #2

Wait, but the skill says "For each approved slice, create a GitHub issue using gh issue create". Let me first check if there's an existing repo/remote to create issues on.

Let me check the git remote first.

Got it - 3 issues, using .done files. The schema fields are clear enough from the PRD so I'll go AFK on that too.

Here's the final breakdown before I create the issues:

#	Title	Type	Blocked by
1	Collector Agent + Project Scaffold	AFK	None
2	Analyzer Agent	AFK	#1
3	Organizer Agent + Pipeline Wiring	AFK	#2

Build DeepSeek V4 Pro DeepSeek

18.8K (2%) · \$0.07 ctrl+p commands

OC | agents-prd.md 拆分为 is x + v

Create issue #3: Organizer + Pipeline Wiring

\$ gh issue create --repo huangjia2019/sdd-in-action --title "Organizer Agent + Pipeline Wiring" --body "\$(cat <<'EOF'

What to build

Implement the **organizer** agent - the final step of the daily pipeline - and wire all three agents together into a complete orchestrated workflow. The organizer reads the analyzer's tagged output and generates a human-readable markdown article at 'knowledge/articles/{date}.md'.

This slice also delivers the pipeline runner that executes collector → analyzer → organizer in sequence, and handles the open questions from the PRD: upstream failure handling, progress tracking, and rerun strategy.

End-to-end flow: runner triggers → collector → (on success) analyzer → (on success) organizer → report completion status. If any agent fails ('.failed' marker detected), downstream agents are skipped.

Scope

- Organizer agent script with CLI entry point

- Markdown article generator (transforms tagged JSON into readable '.md')

- Pipeline runner/orchestrator script

- Upstream failure propagation: if collector writes '.failed', analyzer and organizer skip

- Progress tracking: clear console output showing which agent is running, skipped, or failed

- Rerun strategy: '--force' on the runner propagates to all three agents, clearing all markers before starting fresh

Acceptance criteria

- [] Running the organizer reads 'knowledge/tagged/{date}.json' and produces 'knowledge/articles/{date}.md'

- [] The generated '.md' article is well-structured and human-readable (headings, links to repos, labeled tags)

Build DeepSeek V4 Pro DeepSeek

18.8K (2%) · \$0.07 ctrl+p commands

```
OC | agents-prd.md 拆分为 is  x + v

## Blocked by

- Blocked by #2 (needs both collector and analyzer agent implementations)
EOF
)"

https://github.com/huangjia2019/sdd-in-action/issues/3

Thinking: All 3 issues created. Let me summarize for the user.

3 份 issues 已创建完成：



| # | Title                              | Blocked by | URL                                                                                                                              |
|---|------------------------------------|------------|----------------------------------------------------------------------------------------------------------------------------------|
| 1 | Collector Agent + Project Scaffold | None       | <a href="https://github.com/huangjia2019/sdd-in-action/issues/1">#1 (https://github.com/huangjia2019/sdd-in-action/issues/1)</a> |
| 2 | Analyzer Agent                     | #1         | <a href="https://github.com/huangjia2019/sdd-in-action/issues/2">#2 (https://github.com/huangjia2019/sdd-in-action/issues/2)</a> |
| 3 | Organizer Agent + Pipeline Wiring  | #2         | <a href="https://github.com/huangjia2019/sdd-in-action/issues/3">#3 (https://github.com/huangjia2019/sdd-in-action/issues/3)</a> |



拆分逻辑：
- #1 - 项目骨架 + schemas + collector, 建立共享的 idempotency/--force 模块
- #2 - analyzer: 读 raw → 打标签 → 存 tagged, 验证上游契约
- #3 - organizer + 完整编排：生成 .md 文章 + pipeline runner (串行执行、上游失败下游跳过、进度追踪、--force 传播重跑)

Build · DeepSeek V4 Pro · 5m 25s

Build DeepSeek V4 Pro DeepSeek

18.8K (2%) · $0.07 ctrl+p commands
```

阶段 2.5 · 生成数据契约 schema(5 分钟)

任务票里的 Acceptance criteria 是自然语言,描述了每个 Agent 该产出什么、该收到什么。但 collector 传给 analyzer 的数据到底长什么样、字段是否合法。需要一份机器可校验的契约。这一步把验收里的数据约定固化成可执行的 JSON Schema。

提醒：schema 的生成不归 to-tickets 管。to-tickets 只产出任务票；除非你事先用 Matt 的 /prototype 做过原型、把 schema 片段内联进票，否则票里不含 schema。把验收里的数据契约固化成 JSON Schema，是你单独的一步。

提示词：

基于任务票里各 Agent 的 Acceptance criteria 和输入输出字段,为 Agent 之间传递的数据生成可执行的 JSON Schema，存到 specs/schemas/，每个数据契约一份。

产出示例(specs/schemas/raw-item.schema.json —— collector 产出、analyzer 消费)：

Plain Text

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
```

```
"title": "RawItem",
"type": "object",
"required": ["repo", "url", "stars", "captured_at"],
"properties": {
  "repo": {"type": "string"},
  "url": {"type": "string", "format": "uri"},
  "stars": {"type": "integer", "minimum": 0},
  "description": {"type": "string"},
  "captured_at": {"type": "string", "format": "date-time"}
}
```

第 2 份 schema(labeled-item.schema.json)在 raw-item 基础上增加 analyzer 打的标签字段(domain / maturity / relevance)，用 allOf 引用 raw-item 复用字段定义。有了 schema，analyzer 收到 collector 的输出后可以先做一次校验，不合法直接拒绝，问题在边界处暴露，而不是流到下游才崩掉。

阶段 3 · Implement (10 分钟)

从 issue 派生 .opencode/agents/{collector,analyzer,organizer}.md——每个 Agent 配置文件直接引用对应 issue 作为职责说明。

A vs B 对比

维度	A 路	B 路
耗时	10 min	40 min
协作依赖	口头约定	Ticket 的 Blocked by 字段
验收标准	主观判断	Ticket 的 Acceptance criteria checklist
failure 场景	跑一次才发现	Issue 规划时就覆盖

新人理解成本	读 3 个文件	读 PRD + 3 份 ticket
--------	---------	--------------------

多 Agent = 多依赖。依赖必须显式化成 issue 的 depends_on，验收必须显式化成 acceptance——任务票才有真相源。

完成清单

- specs/agents-prd.md（高阶 PRD）
- specs/issues/{01-collector, 02-analyzer, 03-organizer}.md(3 份 ticket · 带 Blocked by + Acceptance criteria)
- specs/schemas/*.json(2 份可执行 schema · 单独一步生成,见阶段 2.5)
- .opencode/agents/{collector,analyzer,organizer}.md（从 issue 派生）

下一节

第 4 节 Skills 能力封装 —— 三个 Agent 的骨架有了。但 collector 真正的“技能”藏在 skills/github-trending/SKILL.md 里。下节引入 **writing-great-skills**，让 AI 用 SDD 方式帮你从零聊出一份可复用 SKILL.md。这是 Week 1 的收官节，敬请期待。